TY ECE AIML A

Game Library & Recommendation System

Course: Database Management System

By

Aadarsh Mohanan 1032231417

Akshat sharma 1032240204

ECE AIML SEC A, G1

Presentation flow

1. **Introduction**
   1. Brief overview of the project and problem statement
2. **Project Aims and Objectives**
   1. Key goals and intended outcomes
3. **Background & Existing System**
   1. Current situation and its limitations
4. **Proposed System**
   1. Features and improvements over the existing system
5. **Software Requirements & Tools Used**
   1. Hardware/software specifications
   2. Development tools, frameworks, and database
6. **System Design**
   1. **Front-End Design** (UI/UX, user interaction flow)
   2. **Back-End Design** (server logic, database connectivity, APIs)
   3. **ER Diagram** (database relationships)
   4. **Table Design (Schema)**
7. **System Implementation**
   1. Integration of front-end, back-end, and database
   2. Key modules and workflows
8. **Results & Discussion**
   1. Outputs, screenshots of the website, and performance analysis
9. **Applications**
   1. Real-world usage and industry relevance
10. **Conclusion & Future Scope**
    1. Summary and possible extensions
11. **Individual Contributions**
    1. Contribution Summary
12. **References**

    Sources, research papers, tools consulted

Introduction

- Title: Game Library & Recommendation System
- Overview: A full-stack, data-driven web application for video game discovery and management.
- Problem Statement:
  - For Users: Difficulty in discovering games that match specific tastes, and the lack of a simple platform for reviews, wishlists, and personalized recommendations.
  - For Developers: A high barrier to entry for showcasing projects and receiving direct user feedback and performance analytics.
  - For System Design: Monolithic apps where a single, heavy task (like AI recommendations) can slow down the entire system

Introduction

- Our Solution:

- A streamlined, single platform with a decoupled microservice architecture to serve all three needs.

Project Aims and Objectives

Key Goals:

- To Design a 3NF Relational Database in MySQL to manage all user, game, review, and wishlist data.
- To Develop a Microservice Architecture using Flask to decouple the main web API from the AI recommendation engine.
- To Implement Role-Based Access Control (RBAC) for three distinct roles: User, Developer, and Admin.
- To Create a Developer Dashboard allowing creators to upload, manage, and view analytics for their own games.
- To Build an Advanced Content-Based AI Engine (scikit-learn, pandas) that:
  - Uses feature weighting (giving 3x importance to genre).
  - Creates a composite user-taste profile from high-rated reviews and wishlisted items.
  - Handles the cold-start problem for new users.

Background & Existing System

- Current Situation & Inspiration:
  - Platforms like Steam, GOG, and itch.io dominate the market.
  - They validate the core architecture (API, DB, Client).
  - However, they are often complex, closed-ecosystems, or lack good personalized (not just "popular") recommendations.

Background & Existing System

- Our Approach:
  - We implement a modern, lightweight version of this architecture.
  - Key Differentiator: We use a decoupled AI engine (ai_server.py) as a separate microservice. This is a standard industry practice for scalability—it ensures that even if the AI calculations are slow, they will never block the main user-facing API.

Proposed System

- Our platform provides distinct features for each user role:
  - User Role
  - Developer Role
  - Admin Role
- For Users (User Role):
  - Secure Login & Registration (using werkzeug password hashing).
  - Dynamic Game Library with search and multi-select filters.
  - Full Review System (submit ratings and text reviews).
  - Wishlist System (add/remove games).
  - AI-Powered Personalized Recommendations.

Proposed System

- For Developers (Developer Role):
  - Simple account upgrade to "Developer" status.
  - Full Developer Dashboard to:
  - Upload new games to the platform.
  - Edit or Delete their own games.
  - View an Analytics tab (Avg. Rating, Total Wishlists) for their projects.
- For Administration (Admin Role):
  - Has all Developer privileges.
  - Can delete any game on the platform, regardless of the owner.

Software & Tools

- Frontend (Client-Side)
  - HTML
  - Tailwind CSS
  - JavaScript
- Database
  - MySQL

Software & Tools

- Backend (Microservices)
  - Main API Server (app.py)
  - Flask (Python)
  - mysql-connector-python
  - werkzeug.security
  - AI Recommendation Server (ai_server.py)
  - Flask (Python)
  - scikit-learn
  - pandas
  - numpy

System Design

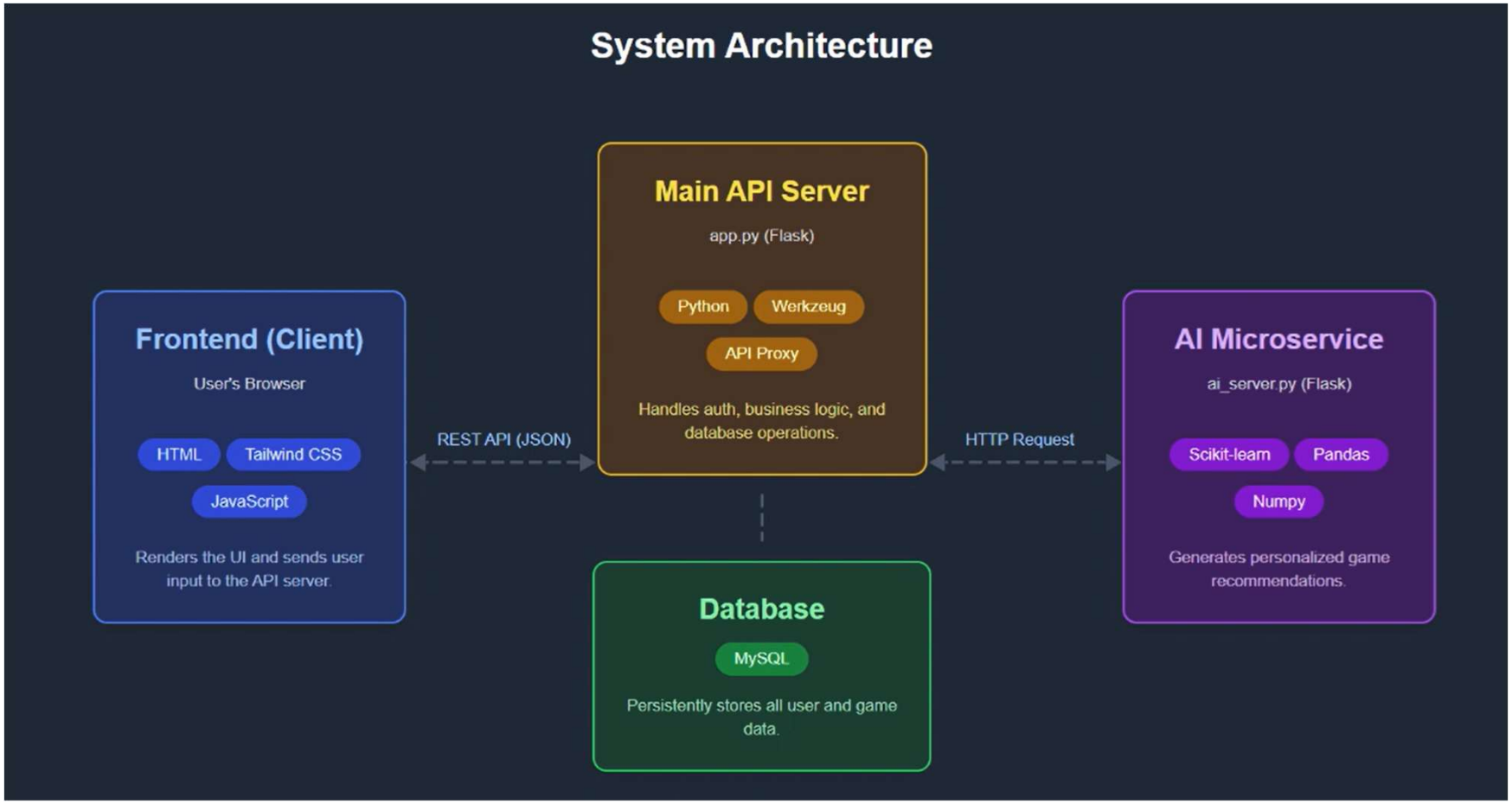
Our system is split into two main backend services:

- Frontend (index.html):
  - A responsive, modal-driven UI. All actions (Login, Register, Game Details, Add Game) happen in popups, creating a smooth, single-page-application feel.

System Design

- Backend (Microservices):
  - Main API Server (app.py):
    - Handles all business logic, user authentication (RBAC), and database CRUD operations.
    - Acts as a proxy to the AI server. The client never talks to the AI server directly.
  - AI Server (ai_server.py):
    - Runs on a separate port (5001).
    - Its only job is to receive a user_id, compute recommendations, and return a JSON list of games.

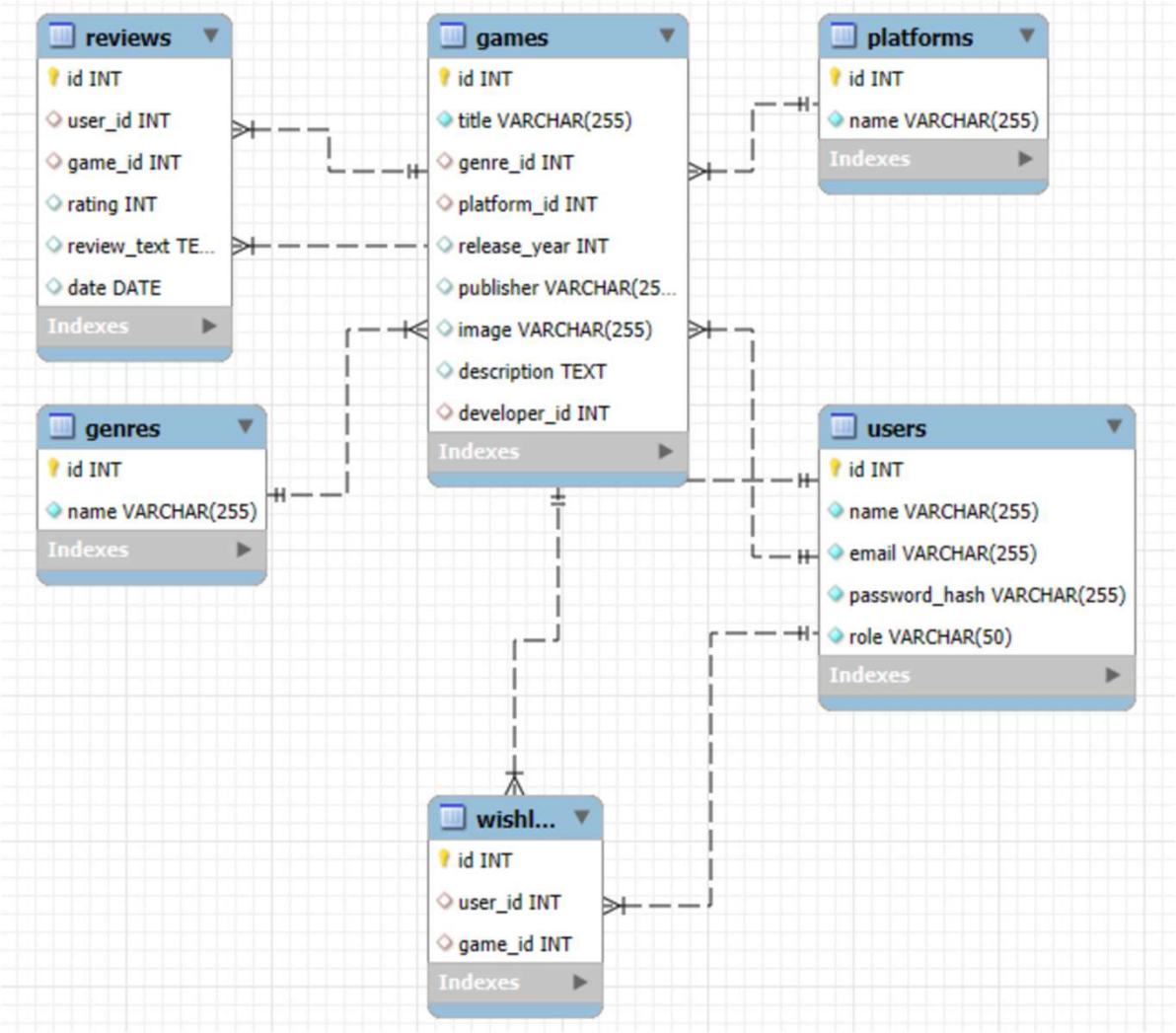
System Design



System Design

ER Diagram

.



System Design

- Our final game_library.sql schema is more advanced than the original ERD.
- Key Additions:
  - A Wishlist table (M:M relationship between Users and Games).
  - A role column in the Users table.
  - A developer_id (FK) in the Games table, linking to Users(id).

System Design

- Table Schema:
  - Users (id (PK), name, email, password_hash, role)
  - Games (id (PK), title, release_year, publisher, description, image, genre_id (FK), platform_id (FK), developer_id (FK))
  - Reviews (id (PK), user_id (FK), game_id (FK), rating, review_text, date)
  - Wishlist (id (PK), user_id (FK), game_id (FK))
  - Genres (id (PK), name)
  - Platforms (id (PK), name)

Implementation

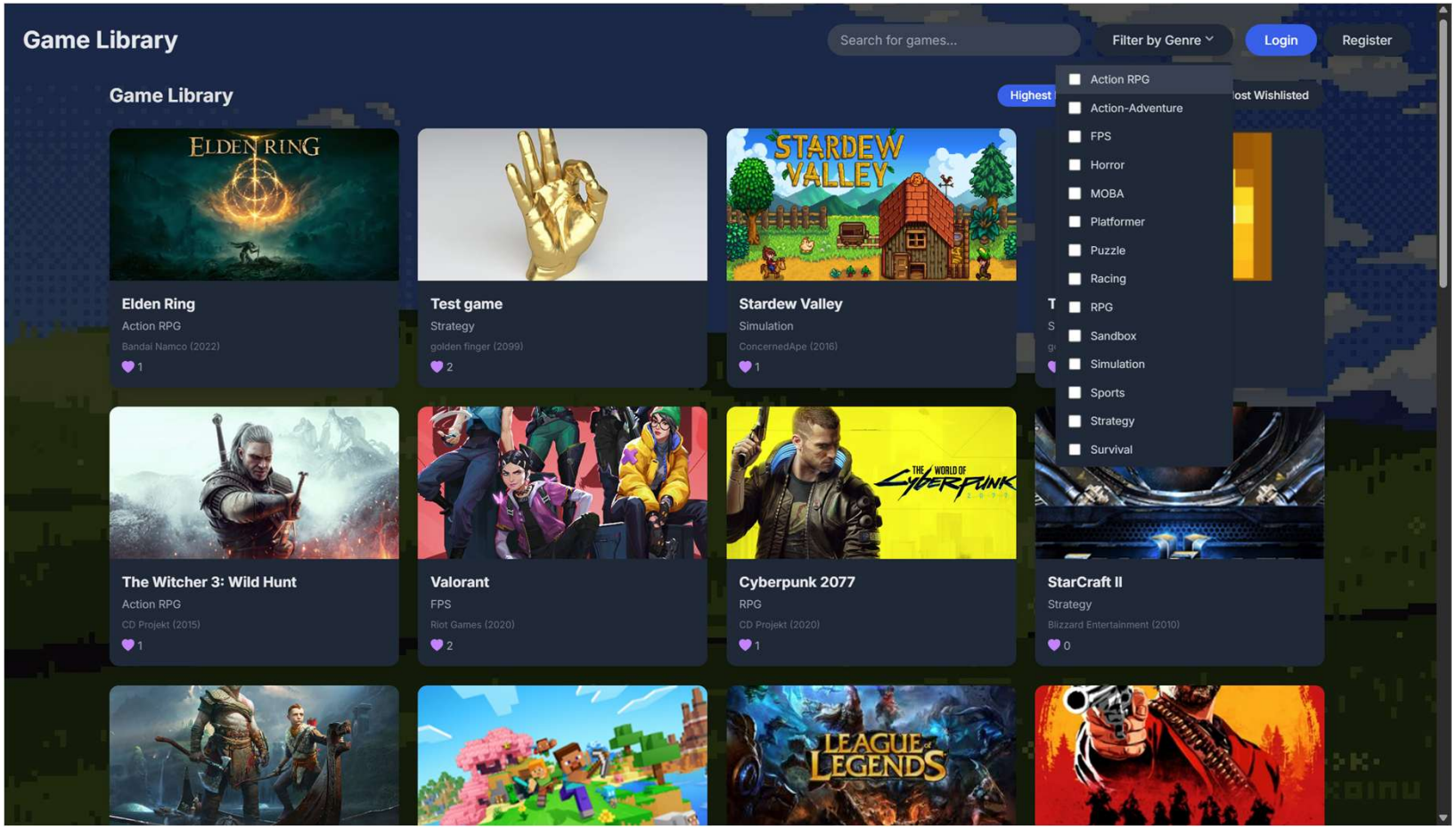
- Integration:
  - The JS Frontend sends a fetch request to the Main API (e.g., GET /api/recommendations/1).
  - The Main API (app.py) receives this, then makes its own server-to-server request to the AI Server (GET http://127.0.0.1:5001/api/recommendations/1).
  - The AI Server (ai_server.py) runs the ML model and returns JSON to the Main API.
  - The Main API returns that JSON to the JS Frontend, which renders the games.

Implementation

- Key Module: The AI Engine (recommendation_system.py)
  - Feature Weighting: Creates a tags string: (genre * 3) + platform + publisher.
  - Vectorizing: Uses CountVectorizer (from sklearn) to convert text tags into numerical vectors.
  - Composite Profile: Creates a single "taste vector" for the user by averaging (np.mean) the vectors of all games they have:
    - Rated 4+ stars
    - …OR added to their Wishlist.
  - Similarity: Uses cosine_similarity to find games that most closely match the user's unique taste vector.

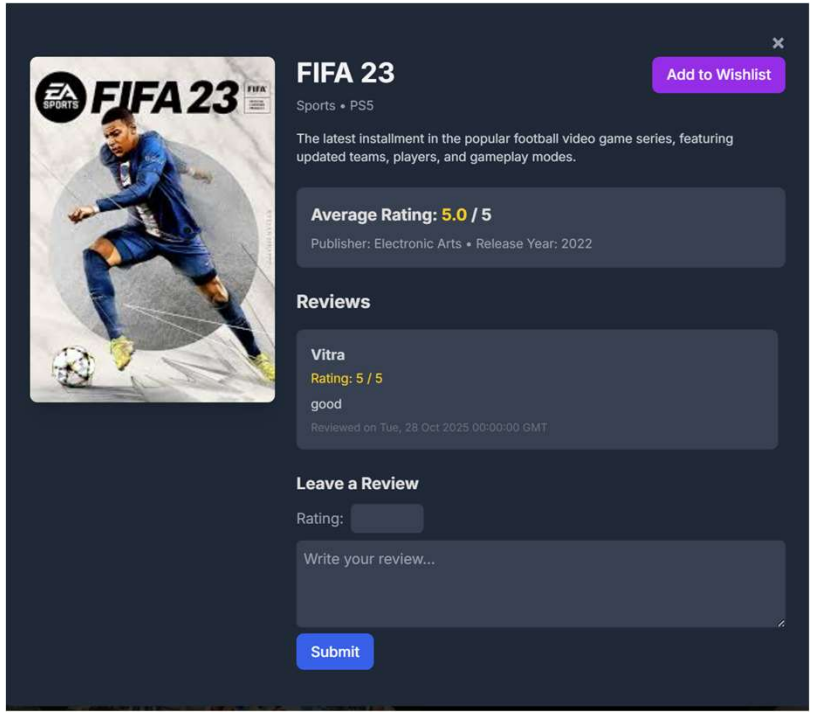
Results & Discussion

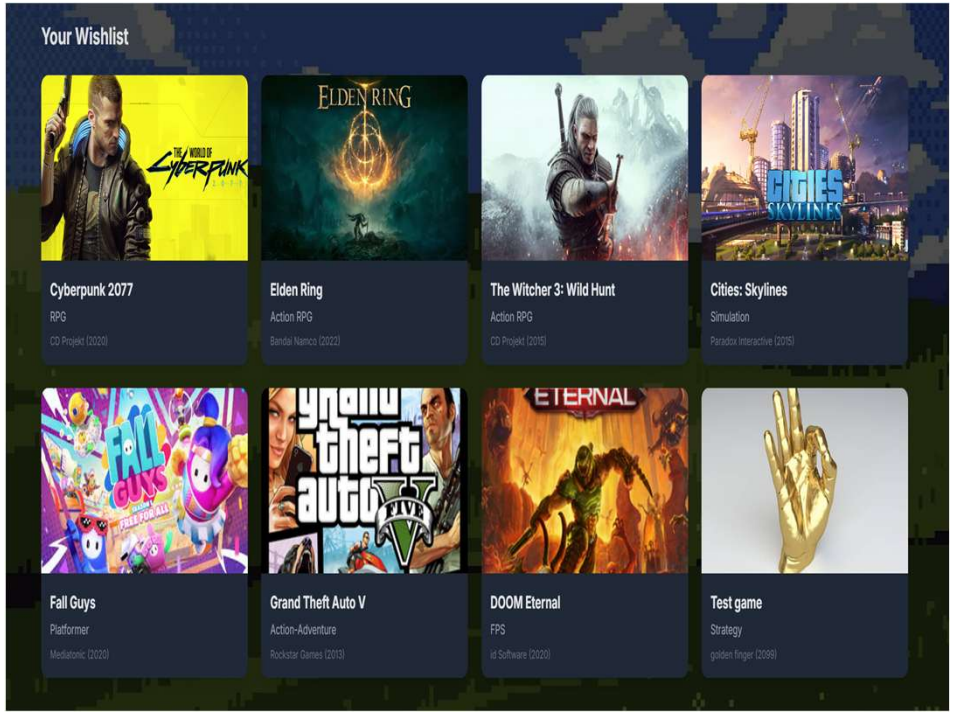
- Main Library & Dynamic Filter:



Results & Discussion

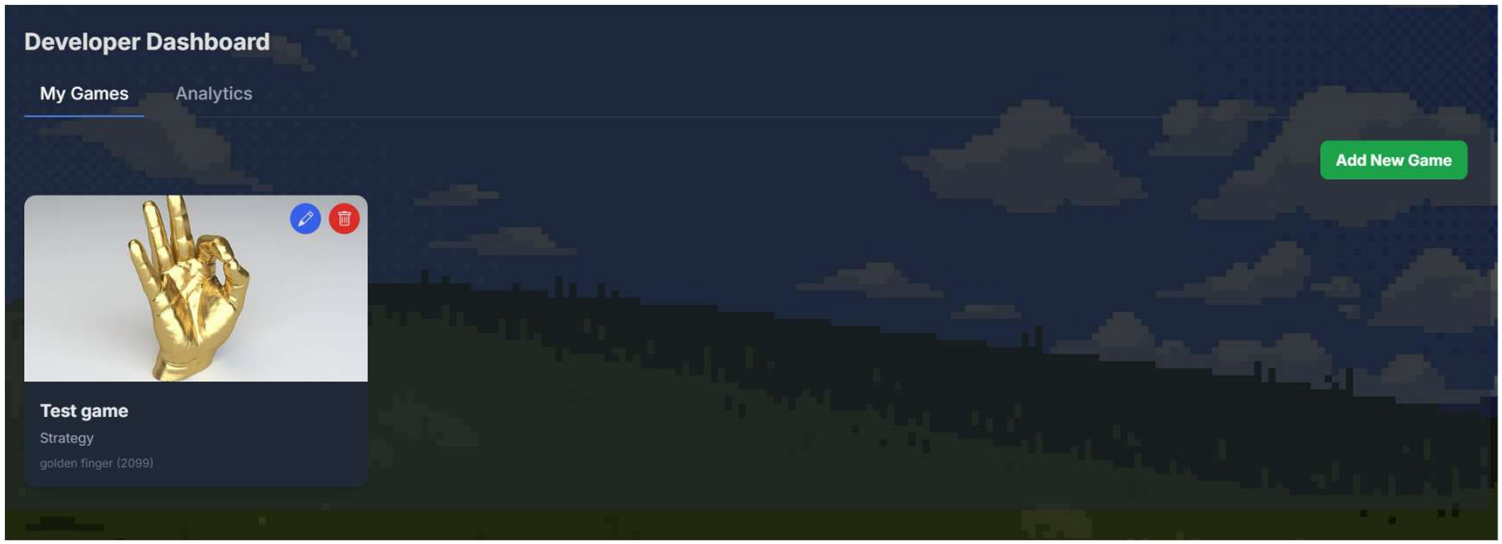
- Modal-Driven UI & Wishlist:

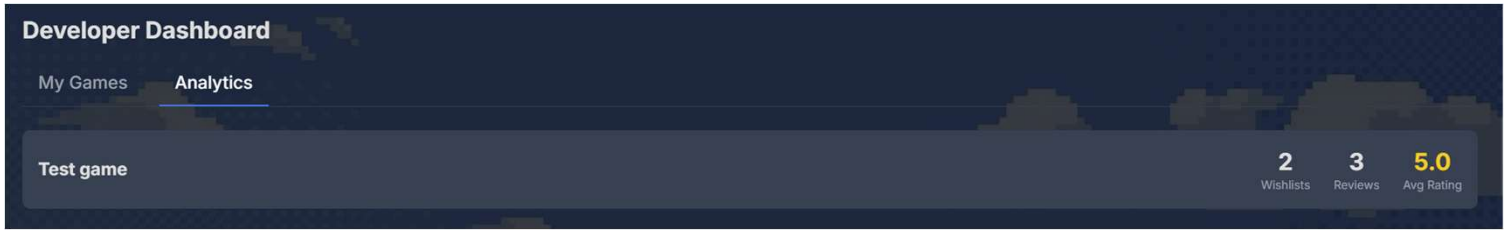




Results & Discussion

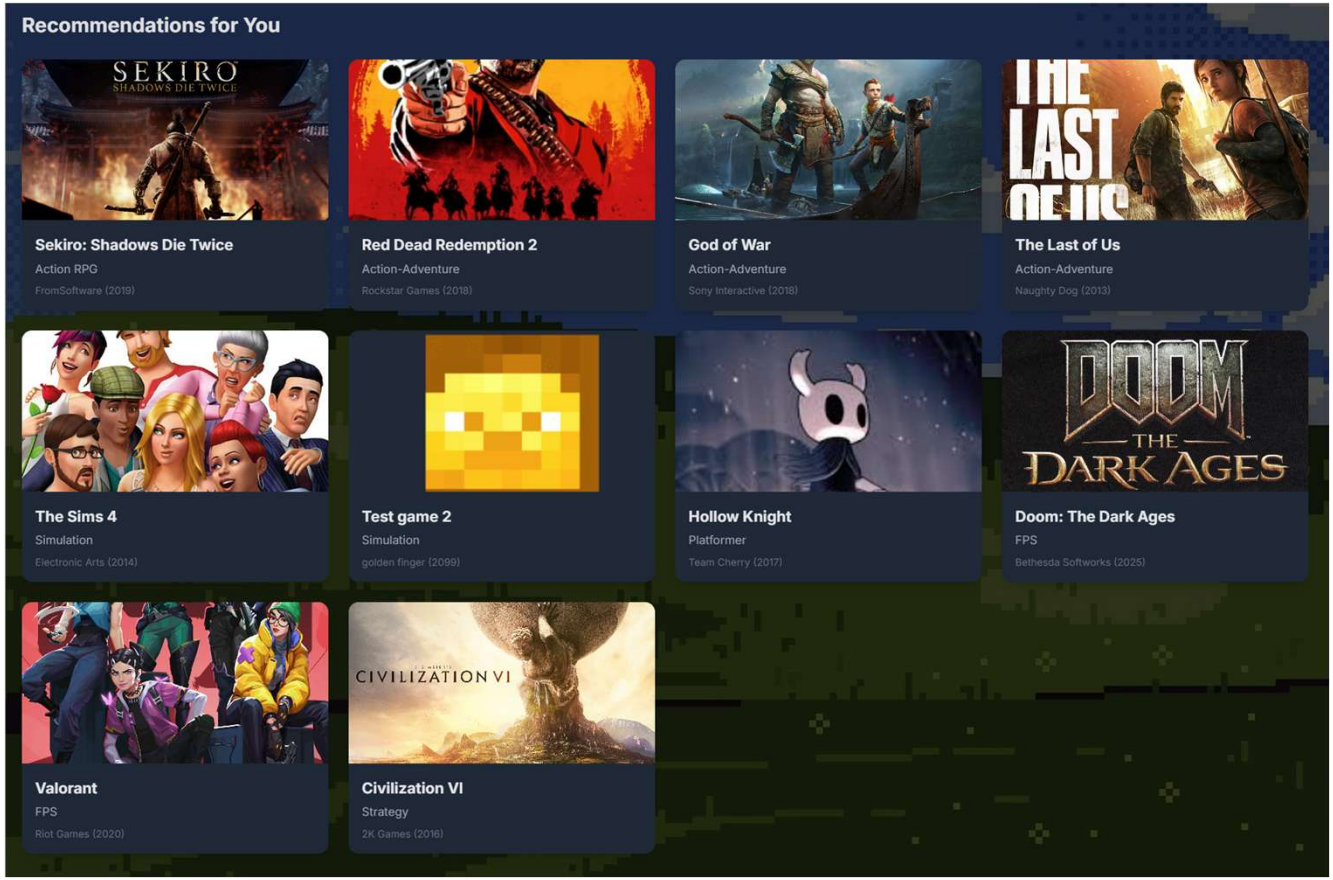
- Developer Dashboard & Analytics





Results & Discussion

- AI-Powered Recommendations



Real-World Applications

- Educational Tool: A complete, self-contained project demonstrating a full-stack, microservice, and ML-integrated application.
- Independent Developer Platform: Can be deployed as a live platform for indie developers to host projects, share their work, and get direct user feedback and analytics.
- Community Platform: Can be scaled into a larger, niche, community-driven review site for specific game genres.

Conclusion & Future Scope

Conclusion:

- We successfully designed and implemented a sophisticated, full-stack application, not just a simple website.
- By adopting a microservice architecture, we cleanly separated the main API from the complex AI engine, ensuring scalability.
- The project fully integrates a MySQL database, a Flask backend with Role-Based Access Control, and a dynamic Tailwind CSS/JavaScript frontend.
- We created a complete ecosystem for both users and game developers.

Conclusion & Future Scope

Future Scope:

- Implement Collaborative Filtering: Create a hybrid AI model that also recommends games based on "users similar to you also liked…"
- Implement JWT Authentication: Move from the current login state to a more secure, stateless JSON Web Token (JWT) system.
- Build Full User Profile Pages: Allow users to see their review history and manage their profile.
- Allow editing/deleting reviews.

Individual Contributions

This project was a balanced partnership, with work partitioned by architectural domain.

- Aadarsh Mohanan (Lead Frontend & System Architect)
  - Frontend Architect (index.html): Designed the complete UI/UX (HTML/Tailwind) and engineered all dynamic JavaScript logic for modals and API calls.
  - API & Security (app.py): Built the core Flask server and the critical User Authentication (werkzeug) & Role-Based Access Control (RBAC) systems.
  - System Integration: Engineered the API proxy in app.py to securely connect the frontend, main API, and AI microservice.
- Akshat Sharma (Lead Data & AI Engineer)
  - Database Architect (game_library.sql): Designed and implemented the complete 3NF relational schema, constraints, and data scripts in MySQL.
  - Data API (app.py): Built all data-facing API endpoints (Game/Review/Wishlist CRUD) and the backend for the Developer Dashboard & analytics.
  - AI Engineer (ai_server.py): Built the dedicated AI microservice and the ML pipeline (Scikit-learn, Pandas) using feature weighting & composite user profiles.

References

[1] P. Py, "Flask web framework," The Pallets Project, Flask Documentation. [Online]. Available: https://flask.palletsprojects.com/. (accessed: September, 2025).

[2] Oracle, "MySQL documentation," Oracle Documentation. [Online]. Available: https://dev.mysql.com/doc/. (accessed: September, 2025).

[3] Mozilla, "HTML: HyperText Markup Language," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML. (accessed: September, 2025).

[4] The pandas development team, "pandas-dev/pandas: Pandas," Zenodo, 2020. [Online]. Available: https://pandas.pydata.org/about/citing.html. (accessed: September, 2025).

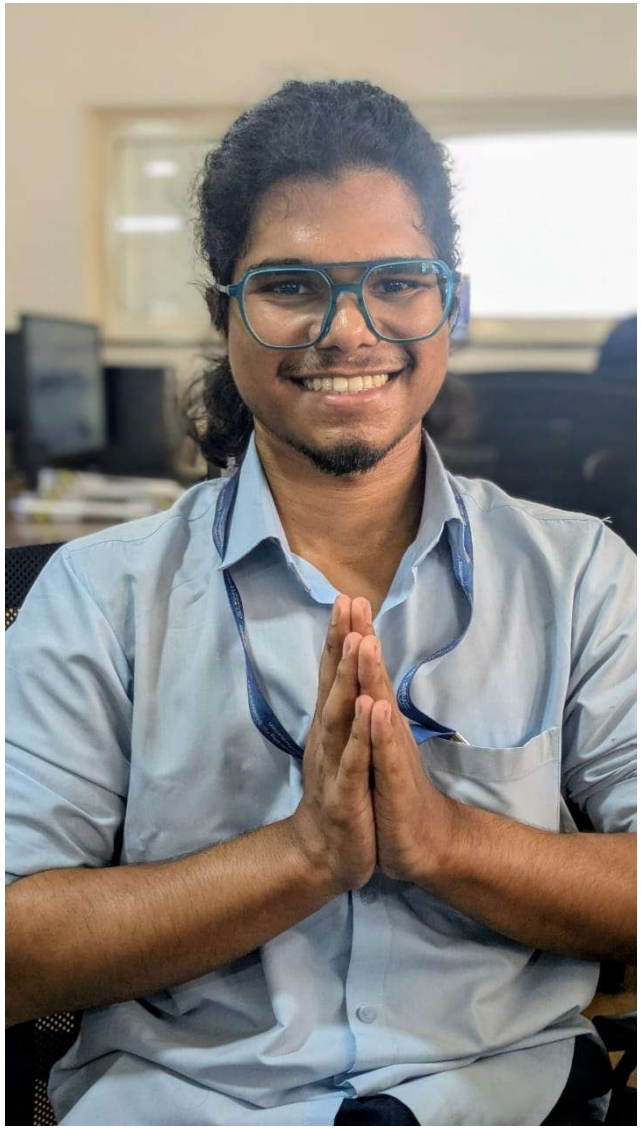
[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[6] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020. [Online]. DOI: 10.1038/s41586-020-2649-2.

[7] G. van Rossum, "Python tutorial, Technical Report CS-R9526," Centrum voor Wiskunde en Informatica (CWI), Amsterdam, May 1995.

[8] Tailwind Labs, "Installation," Tailwind CSS Documentation. [Online]. Available: https://tailwindcss.com/docs/installation. (accessed: September, 2025).

[9] The Pallets Project, "Werkzeug: The Python WSGI Utility Library," Werkzeug Documentation. [Online]. Available: https://werkzeug.palletsprojects.com/. (accessed: September, 2025).



Thank You